

Algoritmos e Estruturas de Dados

Turma: BCC ▪ **Semestre:** 2026.2

Rede de distribuição

---

1 Problema

Considere uma rede de distribuição de mercadorias, composta por depósitos distribuídos geograficamente em um território amplo (estado, país, mundo). Periodicamente, a rede sofre uma reavaliação e é atualizada com a criação ou o fechamento de depósitos.

Cada depósito tem um lucro associado, positivo ou negativo (no segundo caso, o depósito está dando prejuízo), e esse lucro é atualizado a cada período de avaliação. A rede tem uma tolerância mínima de lucro T, que pode ser negativa: depósitos com lucro inferior a T são fechados. Um novo depósito introduzido na rede começa com lucro L informado pela empresa, oriundo de um período anterior de avaliação.

Os depósitos usam rotas de transporte específicas entre eles, e cada rota tem um custo associado (combustível / pedágio). Por simplicidade, os custos são números inteiros positivos (não há custo negativo). As rotas têm direção: se há uma rota do depósito u ao depósito v, não significa que há uma rota de v a u.

Observe que a distância geográfica não tem a ver com os custos das rotas. Cada depósito tem um par de coordenadas inteiras, o que permite calcular a distância entre dois depósitos quaisquer. Considere que a distância geográfica, aqui, é a distância em linha reta, como se os depósitos estivessem em um plano cartesiano.

A gestão da rede exige a realização das operações seguintes, detalhadas mais adiante:

- dado um depósito s, obter a lista de custos de transporte de s aos demais depósitos da rede;
- obter o par de depósitos geograficamente mais próximos;
- remover os depósitos com lucro inferior à tolerância;
- inserir um novo depósito;
- mudar o lucro de um depósito;
- obter o intervalo de períodos consecutivos com a maior soma de lucros, dentre os períodos já registrados;
- obter o período de prosperidade corrente da rede.

A rede conta com milhares de depósitos, então é relevante que essas operações sejam executadas com eficiência.

1.1 Evolução da rede

Os períodos são identificados por inteiros consecutivos, começando em 1. As operações só são realizadas ao término de cada período, em lotes:

- o período 1 começa com um *setup* inicial da rede, e seu lucro é calculado a partir dessa configuração;
- no k-ésimo período, $k > 1$, o lote de operações é executado (término do período anterior) e, em seguida, o lucro do período é calculado sobre a rede atualizada.

Cada período mantém registrado seu lucro total L_k, a soma dos lucros de todos os depósitos da rede naquele período (definição precisa mais adiante).

1.2 Detalhamento das operações

Um depósito é *vivo* desde que é criado (no *setup* ou por uma inserção) até ser removido. Cada depósito tem um identificador inteiro positivo, único e nunca reutilizado, mesmo depois que o depósito é removido (isso é algo a assumir da entrada, não a gerenciar: veja as garantias sobre a entrada).

As operações de um lote são executadas na ordem em que aparecem. Cada consulta enxerga a rede no estado em que ela se encontra naquele ponto do lote.

- `DIST` s: o depósito s está vivo. Imprime uma linha com o menor custo de transporte de s a cada um dos demais depósitos vivos, em ordem crescente de identificador, separados por um espaço. O custo de um caminho é a soma dos custos das rotas percorridas, respeitando a direção das rotas. Se um depósito não é alcançável a partir de s, imprime-se -1 em seu lugar. Se s é o único depósito vivo, imprime-se uma linha vazia.
- `PAR`: imprime uma linha `a b d`, onde `a` e `b` (`a` < `b`) são os identificadores dos dois depósitos vivos geograficamente mais próximos e `d` é a distância euclidiana entre eles, arredondada para cima: o menor inteiro t tal que $t^2 \geq (x_a - x_b)^2 + (y_a - y_b)^2$. Calcule-o com aritmética inteira.
- `REMOVER`: fecha todos os depósitos vivos cujo lucro é estritamente menor que a tolerância T, removendo também as rotas que chegam a eles ou saem deles. Imprime uma linha com a quantidade de depósitos removidos. Depósitos só são fechados por esta operação: um depósito cujo lucro caiu abaixo de T continua vivo, e continua entrando na soma do período, até que `REMOVER` seja executada.
- `INSERIR` id x y L r: cria o depósito id, de coordenadas (x, y) e lucro L, seguido de r linhas `u v c`, cada uma descrevendo uma rota de u a v com custo c em que um dos extremos é id e o outro é um depósito vivo. Não imprime nada.
- `LUCRO` id L: atribui o valor L ao lucro do depósito vivo id (o valor anterior é substituído, não somado). L pode ser menor que T. Não imprime nada.
- `MELHOR`: imprime uma linha `i j S` com o intervalo de períodos consecutivos de maior soma de lucros, conforme a próxima subseção.
- `PROSPERIDADE`: imprime uma linha descrevendo o período de prosperidade corrente, conforme a subseção *Pilha de prosperidade*.

1.3 Lucro do período e melhor intervalo

O lucro do período k, denotado L_k, é a soma dos lucros de todos os depósitos vivos no instante em que o lote do período k termina (para $k = 1$, logo após o *setup*). Se não há depósito vivo, $L_k = 0$. Depósitos abaixo da tolerância que ainda não foram removidos entram na soma. Uma vez calculado, L_k não é mais alterado.

A operação `MELHOR`, executada no lote do período k, considera apenas os períodos já registrados, $1, \ldots, k-1$ (o lucro L_k só é calculado ao fim do lote). Ela determina o intervalo $[i, j]$, com $1 \leq i \leq j \leq k-1$, que maximiza

$$S = L_i + L_{i+1} + \cdots + L_j.$$

O intervalo é sempre não vazio, mesmo que todos os L sejam negativos. Em caso de empate, escolhe-se o intervalo de menor i e, persistindo o empate, o de menor j. A saída é `i j S`.

1.4 Pilha de prosperidade

Além do histórico de lucros, a rede mantém uma *pilha de prosperidade*, que resume quanto do lucro acumulado ainda não foi compensado por prejuízos posteriores. Cada elemento da pilha é um par (j, r): j é um período em que houve lucro positivo e $r > 0$ é o lucro *residual* desse período, isto é, a parte dele que ainda não foi absorvida por prejuízos. A base da pilha guarda o ganho mais antigo, e o topo, o mais recente.

A pilha começa vazia e é atualizada exatamente uma vez por período, logo após o cálculo de L_k, de acordo com o sinal de L_k:

- se $L_k > 0$, empilha-se (k, L_k);
- se $L_k = 0$, a pilha não muda;
- se $L_k < 0$, seja $d = -L_k$ o prejuízo a compensar. Enquanto $d > 0$ e a pilha não está vazia, olha-se o topo (j, r): se $r > d$, o topo é substituído por $(j, r - d)$ e d passa a valer 0; caso contrário ($r \leq d$), o topo é desempilhado e d passa a valer $d - r$. Se a pilha esvazia e ainda sobra prejuízo, a sobra é descartada: a pilha permanece vazia até o próximo período com lucro positivo.

Note que o elemento substituído mantém o índice j do período em que o ganho ocorreu, e que um prejuízo que consome exatamente o topo o desempilha sem empilhar nada (os resíduos são sempre estritamente positivos).

Seja S a soma dos resíduos da pilha. Se a pilha não está vazia e b é o período do elemento da base, então, após o registro do período k,

$$S = L_b + L_{b+1} + \cdots + L_k.$$

O intervalo $[b, k]$ é o *período de prosperidade corrente*: formado por períodos consecutivos, nele o lucro acumulado nunca foi totalmente consumido por prejuízos, e o seu lucro total é S. Quando a pilha está vazia, não há período de prosperidade corrente.

A operação `PROSPERIDADE`, executada no lote do período k, considera a pilha após o registro do período $k - 1$. Se a pilha está vazia, imprime `0`. Caso contrário, imprime uma linha `h b s i r`, onde `h` é a altura da pilha, `b` é o período do elemento da base, `s` é a soma dos resíduos (igual a $L_b + \cdots + L_{k-1}$) e `(i, r)` é o elemento do topo: `i` é o período mais recente cujo ganho ainda não foi totalmente compensado e `r` é o seu lucro residual.

1.5 Formato da entrada e da saída

A entrada é lida da entrada padrão e a saída é escrita na saída padrão. Todos os valores são inteiros, separados por espaços ou quebras de linha, exceto os nomes das operações.

```
n m T
id x y L            (n linhas: depósitos do setup inicial)
u v c               (m linhas: rotas do setup inicial)
P                   (número de períodos além do primeiro)
q                   (início do lote do período 2; repete-se P vezes)
operacao            (q linhas; um INSERIR é seguido de suas r rotas,
                     que não contam em q)
```

A tolerância T vale para toda a execução. Os lotes aparecem na ordem dos períodos $2, 3, \ldots, P + 1$. A saída tem uma linha para cada operação `DIST`, `PAR`, `REMOVER`, `MELHOR` e `PROSPERIDADE`, na ordem em que elas são executadas.

Garantias sobre a entrada.

- No *setup* e em toda inserção, o lucro do depósito é maior ou igual a T.
- O identificador de um depósito criado por `INSERIR` nunca foi usado antes, nem por um depósito vivo, nem por um já removido. Assuma isso da entrada: o programa não precisa verificar nem manter o conjunto de identificadores já usados.
- As coordenadas dos depósitos vivos são sempre distintas entre si.
- Não há rotas de um depósito a ele mesmo, e há no máximo uma rota de u a v para cada par ordenado (u, v). Todas as rotas ligam depósitos vivos.
- `DIST` e `LUCRO` se referem a depósitos vivos, e `PAR` só é executada com pelo menos dois depósitos vivos.
- Sempre que `PAR` é executada, o par mais próximo é único, considerando a distância exata (não a arredondada).

Limites.

- $1 \leq id \leq 10^9$; $|x|, |y| \leq 10^6$; $1 \leq c \leq 10^6$; $|L|, |T| \leq 10^9$. Use inteiros de 64 bits.
- Nunca há mais de 5000 depósitos vivos. No *setup*, $m \leq 5n$, e cada `INSERIR` traz $r \leq 20$ rotas.
- $P \leq 10^5$, com no máximo 3×10^5 operações no total, das quais no máximo 200 são `DIST` e 500 são `PAR`.

O limite de tempo é definido pela ferramenta de correção. Espera-se que cada operação seja executada com eficiência compatível com os limites acima.

1.6 Restrições

- O par mais próximo deve ser calculado algoritmicamente a partir das coordenadas dos depósitos vivos no momento da consulta. É proibido manter qualquer estrutura que armazene distâncias entre pares de depósitos (matriz, tabela, *heap* de pares ou equivalente). Guardar o último resultado e reutilizá-lo enquanto o conjunto de depósitos vivos não mudar é permitido.
- A remoção de um depósito deve ser refletida em todas as estruturas que o mencionam.
- Você pode usar as estruturas de dados da biblioteca da linguagem ou implementar as suas. Em ambos os casos, a escolha deve ser documentada, conforme a seção *Documentação*.

1.7 Exemplo

Entrada:

```
4 4 0
1 0 0 10
2 2 2 4
3 3 4 2
4 10 10 6
1 2 7
2 3 2
1 3 12
3 4 1
5
5
```

```
DIST 1
LUCRO 1 -20
PAR
PROSPERIDADE
MELHOR
7
PROSPERIDADE
MELHOR
REMOVER
DIST 2
INSERIR 5 3 3 3 2
2 5 2
5 4 4
DIST 2
PAR
2
PROSPERIDADE
LUCRO 2 -40
4
REMOVER
DIST 3
PROSPERIDADE
MELHOR
2
PROSPERIDADE
MELHOR
```

Saída:

```
7 9 10
2 3 3
1 1 22 1 22
1 1 22
1 1 14 1 14
1 1 22
1
2 3
2 3 2
3 5 1
2 1 29 3 15
1
1 -1
0
1 3 29
1 5 11 5 11
1 3 29
```

Os lucros dos períodos são $L_1 = 22$, $L_2 = -8$, $L_3 = 15$, $L_4 = -29$ e $L_5 = 11$. No período 2, o depósito 1 passa a ter lucro -20 mas continua vivo e entra na soma, o que torna L_2 negativo. Ele só é fechado pela operação `REMOVER` do lote seguinte, e suas rotas desaparecem com ele. Na primeira `PAR`, o par mais próximo é formado pelos depósitos 2 e 3, cuja distância é $\sqrt{5} \approx 2{,}24$, impressa como 3 (arredondada para cima); na segunda, depois da inserção do depósito 5, o par é 3 e 5, a distância 1.

Evolução da pilha de prosperidade (base à esquerda), após o registro de cada período:

k	L_k	pilha
1	22	$[(1, 22)]$
2	-8	$[(1, 14)]$
3	15	$[(1, 14), (3, 15)]$
4	-29	$[\,]$
5	11	$[(5, 11)]$

No período 4, o prejuízo 29 consome o topo $(3, 15)$ e, exatamente, o elemento $(1, 14)$, esvaziando a pilha. A `PROSPERIDADE` do lote 4 imprime `2 1 29 3 15`: altura 2, período de prosperidade começando em 1 com lucro total $22 - 8 + 15 = 29$, e topo $(3, 15)$. Observe que o melhor intervalo de `MELHOR` no lote 6 (1 3 29) continua sendo o histórico, enquanto o período de prosperidade corrente já recomeçou no período 5.

2 Entregáveis

A atividade é dividida em três entregáveis, cada um originando uma nota própria. Cada entregável acrescenta operações ao anterior e pode ser desenvolvido de forma independente, a partir das descrições abaixo, mas reutiliza os módulos dos entregáveis anteriores.

Forma de entrega.

- Cada entregável é um programa completo (lê a entrada padrão e escreve a saída padrão) em **um único arquivo-fonte**, por exemplo `e1.c` ou `e1.py`. Não entregue arquivos auxiliares, como cabeçalhos `.h`, módulos adicionais ou *makefiles*.
- Como cada submissão é um único arquivo, o código dos entregáveis anteriores que você reutilizar é incorporado ao arquivo do entregável seguinte (copiado, com as correções que você julgar necessárias).
- Em cada estágio, somente as operações daquele estágio são exercitadas pelos testes, conforme descrito nas subseções seguintes. A saída é comparada linha a linha, dentro do limite de tempo da ferramenta de correção.
- A leitura do *setup* (cabeçalho, depósitos e rotas) e das linhas de cada operação é comum a todos os estágios, inclusive das rotas, que devem ser lidas mesmo quando o estágio não as usa. Uma leitura incorreta do *setup* invalida todos os testes do estágio.

2.1 Ponto de ancoragem

Cada arquivo deve conter um *ponto de ancoragem* com a ferramenta de correção. A ferramenta executa esse ponto de entrada, que é o responsável por consumir a entrada padrão e por escrever as respostas na saída padrão. Por exemplo: a função `main`, em C e C++; a classe `Main` (com seu método `main`), em Java; a função `main` chamada no bloco `if __name__ == "__main__"`, em Python. Todo o resto do programa (estruturas, funções auxiliares) é chamado a partir dele. Nada além das respostas deve ser escrito na saída padrão.

2.2 Exemplo de ponto de ancoragem

O exemplo a seguir, em Python, é só a casca: lê inteiros da entrada padrão (aqui, o cabeçalho e os depósitos do *setup*) e escreve inteiros na saída padrão. A leitura das rotas, dos lotes e das operações, e todo o tratamento delas, ficam por sua conta.

```
import sys

def main():
    tokens = sys.stdin.read().split()
    n, m, T = int(tokens[0]), int(tokens[1]), int(tokens[2])
    pos = 3
    for _ in range(n):                  # n linhas "id x y L"
        ident, x, y, lucro = (int(t) for t in tokens[pos:pos + 4])
        pos += 4
        print(ident, lucro)             # escreve inteiros em stdout
    # ... rotas, lotes de operações e respostas continuam daqui

if __name__ == "__main__":
    main()
```

2.3 Entregável 1: histórico de períodos

Operações exercitadas. `INSERIR`, `LUCRO`, `MELHOR` e `PROSPERIDADE`. Não aparecem `REMOVER`, `DIST` nem `PAR`. As rotas são lidas, mas podem ser descartadas.

O que se espera.

- um registro dos depósitos com acesso por identificador (os identificadores chegam a 10^9, e há no máximo 5000 depósitos), com o lucro total do período mantido sem varrer todos os depósitos a cada período;
- a pilha de prosperidade, com as regras da subseção correspondente;
- a resposta de `MELHOR` sem reexaminar todo o histórico a cada consulta.

Justifique, na documentação, a estrutura escolhida para o registro.

Tópicos exercitados. Pilha, análise amortizada, programação dinâmica (soma máxima de subvetor consecutivo) e busca por chave.

2.4 Entregável 2: fechamento e proximidade

Operações exercitadas. `REMOVER` e `PAR`, com `INSERIR` e `LUCRO` para que a rede evolua entre as consultas. Não aparecem `MELHOR`, `PROSPERIDADE` nem `DIST`. As rotas são lidas, mas podem ser descartadas.

O que se espera.

- `REMOVER` deve localizar os depósitos abaixo da tolerância sem examinar todos os depósitos vivos a cada chamada, mesmo com lucros que mudam ao longo dos períodos. O registro do Entregável 1 passa a precisar de remoção;
- `PAR` deve ser calculado algoritmicamente, conforme as restrições, com complexidade melhor que a de comparar todos os pares.

Tópicos exercitados. Fila de prioridade com alteração de chave e divisão e conquista.

2.5 Entregável 3: rede de rotas

Operações exercitadas. `DIST`, com `INSERIR` e `REMOVER` para que o grafo evolua (a linha impressa por `REMOVER` também é comparada). Não aparecem `MELHOR`, `PROSPERIDADE` nem `PAR`. As rotas deixam de ser descartadas.

O que se espera.

- um grafo dirigido dinâmico: `INSERIR` acrescenta rotas e a remoção de um depósito elimina as rotas que o tocam;
- `DIST` por um algoritmo de caminhos mínimos adequado a custos positivos, reutilizando a fila de prioridade do Entregável 2;
- a remoção do Entregável 2 deve alcançar o grafo, reutilizando o registro dos depósitos do Entregável 1.

Tópicos exercitados. Grafos, representação esparsa e caminhos mínimos.

2.6 Documentação

A documentação faz parte da avaliação e deve estar no próprio arquivo-fonte, como comentários (ou *docstrings*).

- **Cabeçalho do arquivo:** nome, entregável, e um resumo de uma ou duas linhas de cada estrutura de dados usada e de por que foi escolhida.
- **Cada estrutura de dados:** se é implementada por você, no ponto de sua definição, uma descrição do que cada campo guarda e dos invariantes que devem valer entre as operações. Se vem da biblioteca da linguagem, indique qual é e a complexidade das operações que você usa (com a hipótese, quando houver, por exemplo "esperado" para tabela *hash*).
- **Cada função:** um comentário de cabeçalho que diga claramente (a) o que a função *recebe*: parâmetros, seus significados e as pré-condições; (b) o que ela *faz* e o que devolve, incluindo os efeitos sobre as estruturas que modifica; e, para toda função que realiza uma operação sobre uma estrutura de dados ou uma operação da rede (inclusive o tratamento de cada comando de entrada), (c) a *complexidade de tempo* no pior caso, ou amortizada, com uma linha de justificativa, e a complexidade de espaço adicional quando não for constante.
- As complexidades são expressas em função de n (depósitos vivos), m (rotas vivas), P (períodos registrados) e h (altura da pilha), conforme o caso.

2.7 Composição de cada nota

Os entregáveis valem 50% da nota, e 50% virá de uma entrevista.